

Расчётная работа

Вариант 1.6: Связный граф

Студент: Артем Морозов, гр. 421702

Дата: 25 мая 2025 г.

Цели работы

- Повторить основные положения теории графов.
- Формализовать алгоритм проверки связности графа пошагово, в соответствии с выданным вариантом.

Условие задания

- Формализовать алгоритм проверки связности неориентированного графа.
- Подготовить отчёт в LaTeX.

Теоретические сведения

Граф — совокупность двух множеств: множества вершин и множества рёбер (пар вершин).

Связный граф — граф, в котором между любой парой вершин существует путь, то есть из любой вершины можно добраться до любой другой вершины по рёбрам.

Список смежности — способ представления графа, при котором каждой вершине сопоставляется список её соседей.

Наглядный пример формализации графа в редакторе КВЕ:

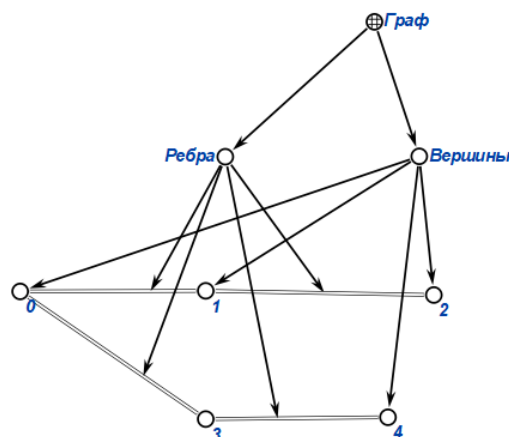


Рис. 1: Рис. 1: Пример неориентированного графа

Однако для наглядности рекомендовано использование более удобного и простого способа создания графа в редакторе КВЕ:

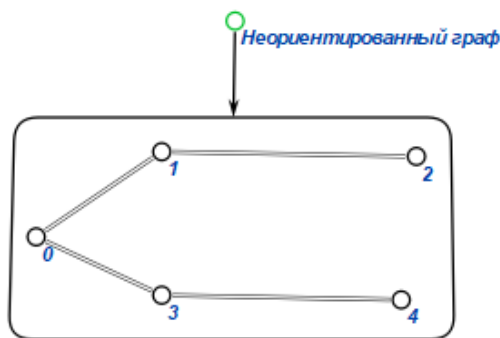


Рис. 2: Рис. 1: Пример неориентированного графа

Формализация алгоритма проверки связности графа

Задача: проверить, является ли заданный неориентированный граф связным.

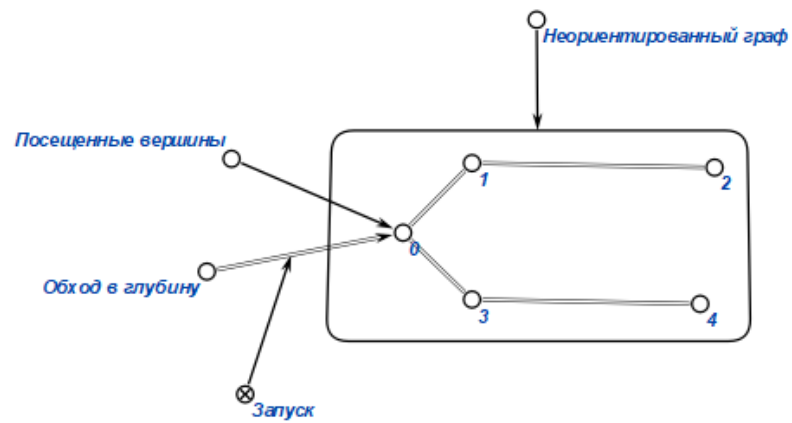
Алгоритм (пошагово)

1. Выбирается произвольная вершина графа в качестве стартовой.
2. Помечаем выбранную вершину как посещённую.
3. Для каждого ребра, инцидентного к текущей вершине, рассматриваем соседнюю вершину.
4. Если соседняя вершина ещё не была посещена, делаем её текущей и повторяем шаги 2–4.
5. Если у текущей вершины не осталось рёбер, ведущих к непосещённым вершинам, возвращаемся к предыдущей вершине и продолжаем обход.
6. После завершения обхода анализируем, были ли посещены все вершины графа:
 - если все вершины посещены, граф является связным;
 - если остались непосещённые вершины, граф не связный.

Иллюстрация работы алгоритма

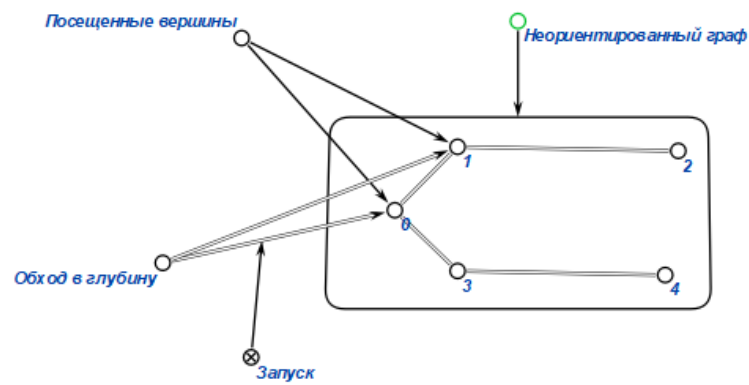
Шаг 1: Выбирается стартовая вершина, например, 0.

Шаг 2: Вершина 0 помечается как посещённая. Рассматриваются все рёбра, исходящие из неё.

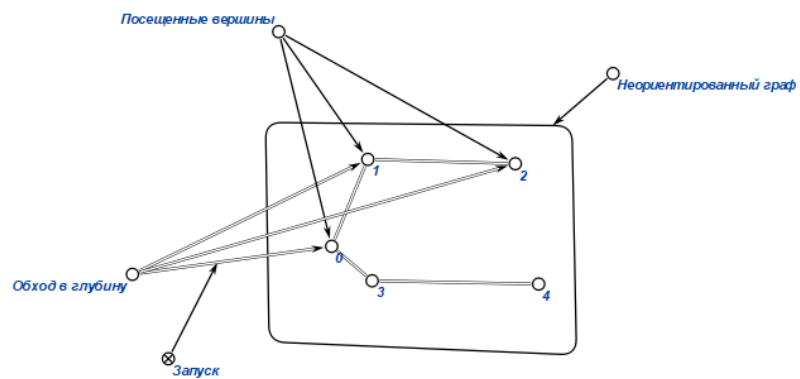


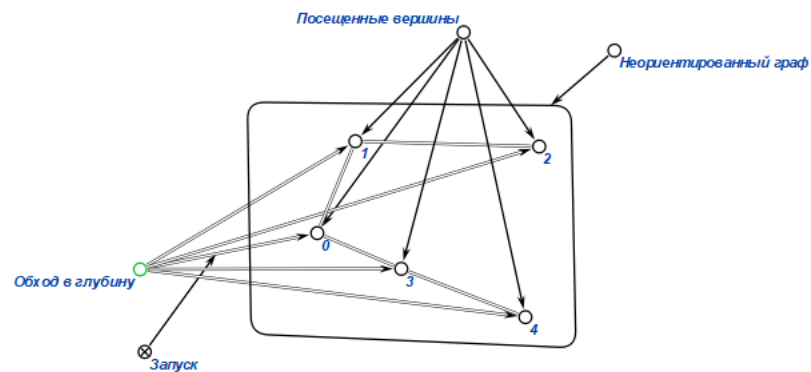
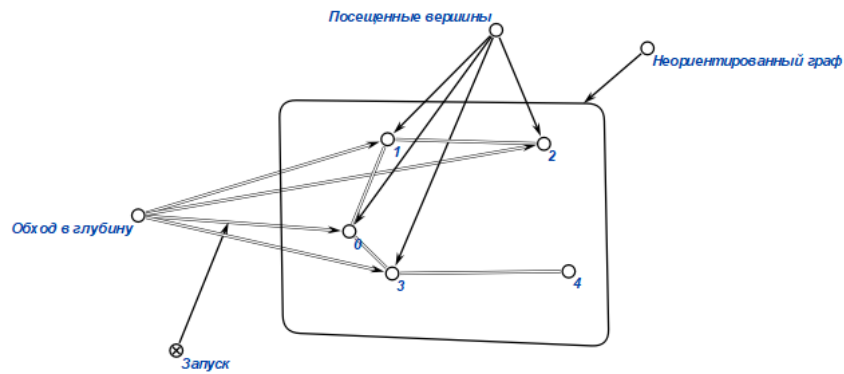
Шаг 3: Для каждого ребра:

- если оно ведёт к непосещённой вершине, переходим к ней, помечаем как посещённую и повторяем процесс;
- если рёбра закончились, возвращаемся к предыдущей вершине.



Шаг 4: Повторяем шаги до тех пор, пока не будут рассмотрены все вершины.

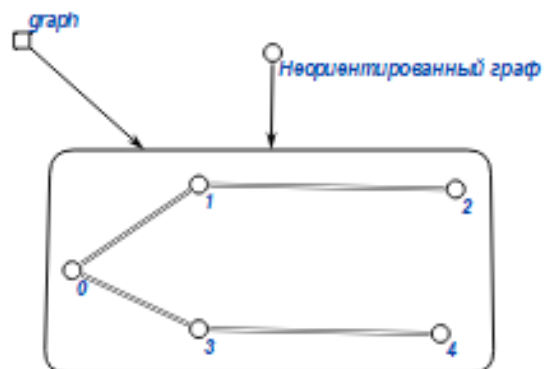




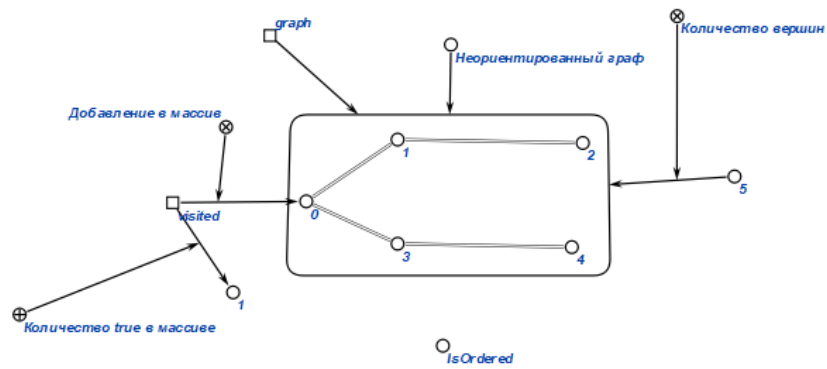
Шаг 5: Если все вершины посещены, граф связный.

Работа алгоритма в коде

Шаг 1: Пользователь задает неориентированный граф:

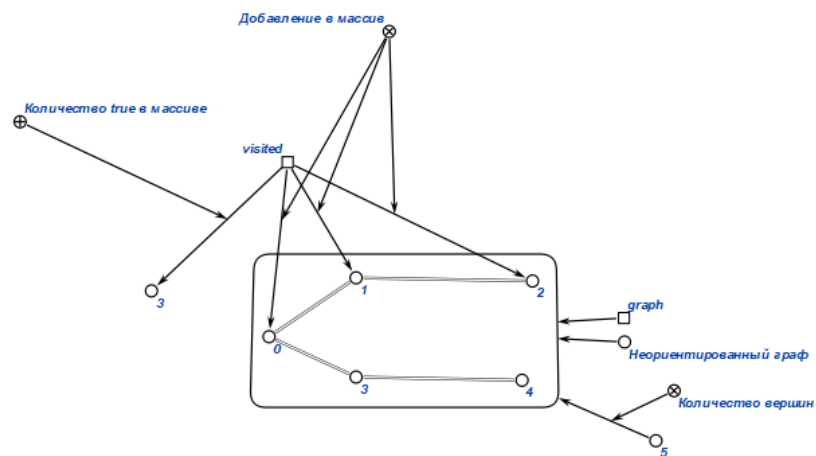
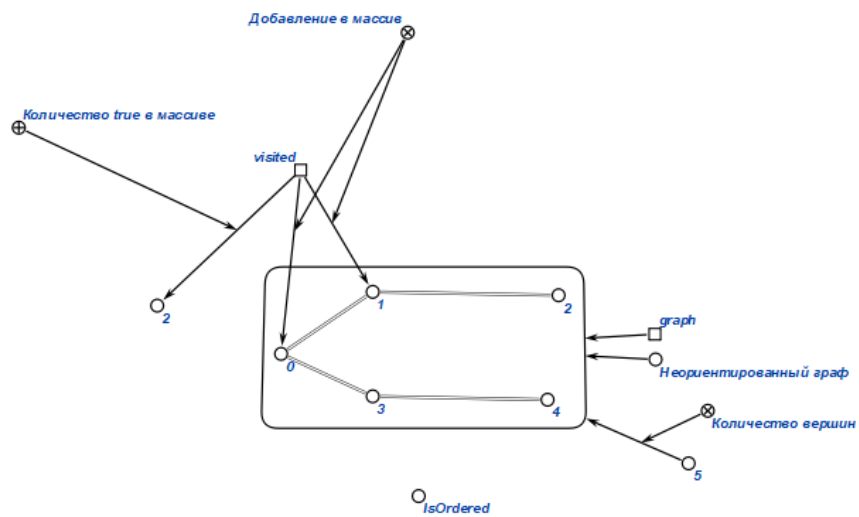


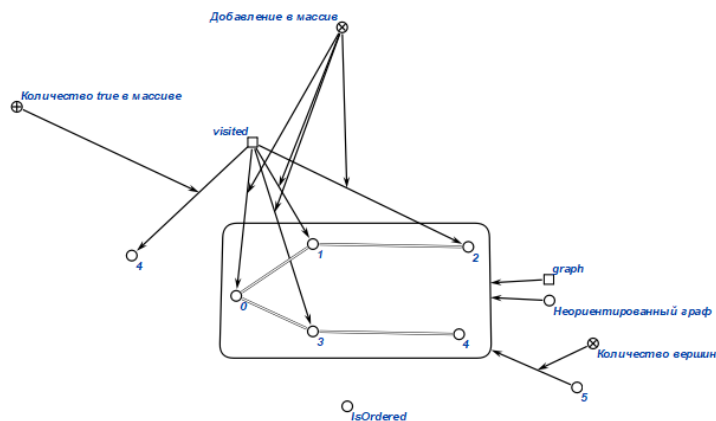
Шаг 2: Создаётся массив `visited[]` размером по количеству вершин, где для каждой вершины хранится информация, была ли она посещена. Так же создается переменная `IsOrdered`, которая покажет, является ли граф ориентированным



Шаг 3: Обход смежных вершин:

- Если вершина ещё не была посещена, вызывается рекурсивный DFS для неё.
- В процессе обхода все достижимые вершины будут помечены как посещённые.

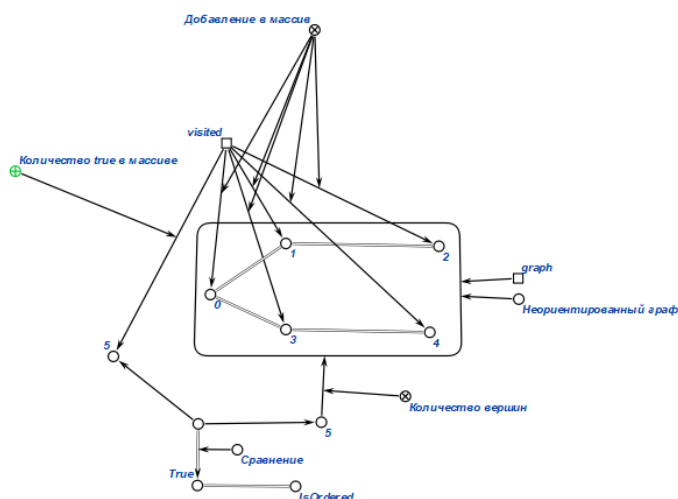




Шаг 4: Проверка результата:

После завершения обхода весь массив `visited[]` проверяется:

- Если число элементов `visited[]` равных `true`, совпадает с общим числом вершин, то из вершины 0 можно добраться до всех остальных, и граф связный.
- Если есть хотя бы один элемент `false`, значит, существует хотя бы одна недостижимая вершина — граф не связный.



Вывод

В ходе работы был формализован и реализован алгоритм проверки связности неориентированного графа. Проведённые тесты подтвердили корректность работы алгоритма.

Используемые источники

1. Работа с графами онлайн [Электронный ресурс] - Режим доступа: graphonline.ru/wiki
2. Граф (математика) [Электронный ресурс] - Режим доступа: <https://ru.wikipedia.org/wiki/Г>
3. Емеличев В. А. и др. Лекции по теории графов. М.: Наука, 1990.